

CashTrack

A Freelance Agency Management Dashboard

Project Report

Project Name	CashTrack — Freelance Agency Management Dashboard
Submitted by	Syed Muhammad Umer Ali 17828
Course	Web Programming Language Lab
Instructor	Faraz Abdul Basit
Department	Computer Science
Date of Submission	September 2026

Abstract

CashTrack is a web-based dashboard that I developed to manage the work and payments of a small freelance team in one place. While working on the idea, I noticed that task management and payment tracking are often handled separately. For example, when a task is completed in one tool, the person managing the work may still have to manually prepare an invoice in another tool. CashTrack connects these steps inside one React application. When work is completed, the task can be used to create an invoice and later update the related team member's payout. The system has two user roles: admin and team member, and each role has access to a different view of the same project data. I developed the application using React with Vite and styled it with Tailwind CSS, using a glassmorphism-style interface. Instead of using an online database in the final version, I used the browser LocalStorage API to store the application data. This was a change I made during development after facing problems with the original Supabase setup. For the analytics section, I also created the charts myself using SVG instead of using a chart library. This report explains how I planned and built the system, the main technical decisions I made, the problems I faced, and what I learned while building my first proper React application.

2. Introduction & Problem Statement

While planning this project, I noticed that many small freelance agencies use several separate tools to manage their daily work. For example, tasks may be managed in Trello, communication may happen through WhatsApp, and payment details may be kept in a spreadsheet or sometimes not properly recorded at all. This setup can work for a very small team, but it becomes harder to manage when there are more clients, team members, tasks, and payments involved.

The main issue is that project work and financial records are usually treated as separate things, even though they are closely connected in a freelance agency. When a task is completed, it can also mean that someone needs to be paid for that work. If the task and payment information are kept in different places, an admin has to match them manually. This can lead to mistakes, such as forgetting to invoice a completed task, recording a payment for the wrong team member, or making it difficult for a team member to know how much they have earned.

The basic idea behind CashTrack was to keep the task information and financial information connected and show the same data according to the user's role. An admin should be able to view clients, tasks, invoices, and payouts, while a team member should only see the work assigned to them and their own earnings. This idea became the main point that guided the structure and features of the application.

3. Project Objectives

Before starting the development, I made a list of the main things I wanted the application to achieve. I divided these goals into functional objectives for the system and learning objectives for myself.

Functional Objectives

- Create a working dashboard where an admin can add and manage clients, assign tasks to team members, and monitor the status of projects from one place.
- Provide each team member with a personal view showing only the tasks assigned to them and their own information, without giving them access to other team members' or clients' financial data.
- Connect completed tasks with the invoicing process so an admin can create an invoice from finished work with minimal manual data entry.
- Maintain a payout history for each team member and update it when an invoice related to their work is marked as paid.
- Show useful information such as earnings, completed tasks, and client activity through an analytics dashboard instead of displaying everything only in tables.

Technical / Learning Objectives

- Learn React by building a complete application with actual features and data flow instead of only following a small tutorial.
- Understand component-based development and learn when a part of the interface should be made into a separate React component.
- Learn how shared state works in React, reduce unnecessary prop drilling, and use React Context when it makes sense for the application.
- Learn how to make practical technical decisions and change an earlier decision when it is no longer suitable for the project, such as changing from Supabase to LocalStorage.

4. Technology Stack & Justification

The final technology stack is quite simple, and I kept it that way on purpose. I selected each technology based on what I needed for the project and the problems I faced while developing it.

React.js (with Vite)

I selected React because CashTrack has several screens that use and update the same data. Tasks are connected to invoices, invoices are connected to payouts, and the resulting information is shown in the analytics section. React's component-based structure made these connections easier for me to organize than managing everything with plain JavaScript and manual DOM updates. I used Vite as the development and build tool because it starts quickly and reloads changes fast, which was helpful while I was repeatedly testing and fixing the application.

Tailwind CSS

I used Tailwind CSS for the styling and gave the dashboard a glassmorphism-style design with transparent panels, soft shadows, and blurred backgrounds. Since the dashboard contains a lot of information, these visual sections helped keep the interface organized and easier to read. Another reason Tailwind was useful for me was that I could write the styling classes directly in the JSX instead of moving between many CSS files while I was still learning how the components and state were connected.

LocalStorage API — and the pivot away from Supabase

At first, I planned to use Supabase for both authentication and database storage, and I also made an early version of the project using it. However, I faced several problems during development. Some requests were blocked by row-level security rules, authentication sessions sometimes caused problems, and environment variables did not always behave the same way after setting up the project again. These issues were taking time away from the actual features I needed to complete. Because the main requirement was to have a reliable application for the final demonstration, I decided to switch to the browser's LocalStorage API. This removed the need for a backend, server configuration, and network requests. The application now stores clients, tasks, invoices, payouts, and login information in LocalStorage through a small data-access layer. This also made the project easier to run on another computer without extra setup.

Custom SVG Charts

For the analytics dashboard, I decided to create the charts myself with SVG components instead of using a library such as Recharts or Chart.js. One reason was to keep the project lightweight and avoid adding another dependency for a small application. More importantly, I wanted to understand how charts are actually built. I worked out the coordinate calculations and scaling needed to place the values inside the SVG viewBox. This was one of the parts of the project where I learned something beyond simply using an existing component.

5. System Architecture & Data Flow

CashTrack uses a component-based structure and is mainly organized around four types of data: clients, tasks, invoices, and users. The related components are responsible for displaying, adding, and editing this information. I kept these UI components focused on presentation and passed the required data and actions to them instead of allowing them to directly access LocalStorage.

State Management with React Context

The shared application data, including the current user, clients, tasks, and invoices, is managed through React Context providers near the top of the component tree. I chose Context instead of a library such as Redux because the amount of state in this project did not require a larger state-management solution. It also gave me a chance to understand how data moves through React. Components can use custom hooks to access the required context, which helped me avoid passing the same information through several levels of components.

The LocalStorage Data Layer

Below the Context providers, I created a small data-access layer. It contains functions such as `getTasks()`, `saveTask()`, `getInvoices()`, and `recordPayout()`, and these functions are responsible for working directly with `window.localStorage`. The data is saved as JSON using separate keys such as `cashtrack_clients`, `cashtrack_tasks`, and `cashtrack_invoices`. Keeping the LocalStorage code in one place made the application easier to manage and meant I could change the way data was stored without editing many different components.

Data Flow, End to End

The overall data flow starts when a component calls an action from the relevant context, such as `completeTask`. The context changes the state in memory and then calls the related data-access function to save the change in LocalStorage. Components that are using that context receive the updated state and re-render automatically. Because of this, the invoice list and analytics section do not need separate instructions to refresh; they update when the shared data changes.

6. Core Features & Functionality

Role-Based Access Control

The application has two roles: admin and team member. After login, the system checks the user's role and shows the appropriate routes and components. The admin can manage clients, view the complete task board, handle invoices, and see the overall analytics. A team member gets a personal task view and can see their own payout history. The data shown to non-admin users is also filtered using their user ID, so the restriction is not only based on hiding buttons or screens.

Task Management with Priorities and Tags

Each task has a priority level such as low, medium, or high. It can also contain tags such as design, copywriting, or revision, along with a due date, assigned team member, and compensation amount. The task board separates work into To Do, In Progress, and Completed. Admins can also filter the tasks by client, team member, or priority, which became useful when I tested the system with a larger set of sample tasks.

The Automated Financial Pipeline

The main purpose of CashTrack is to connect completed work with the payment process. When a task is moved to Completed, it becomes available for invoicing. The admin can select one or more completed tasks for a client and create an invoice. The invoice details, task amounts, and related team member payout information are taken from the task data, so the same information does not need to be entered again manually.

Analytics Dashboard

The analytics dashboard displays information from the same data used by the rest of the application. It includes a line chart for earnings over time, a bar chart for task completion by team member, and summary cards showing values such as total revenue, unpaid invoices, and active tasks. Since these values are calculated from the LocalStorage data used by the task and invoice sections, the dashboard stays connected to the current records.

7. The Automated Workflow (Step-by-Step)

To explain how the main features work together, the following steps show what happens to a task from the time a client is added until the related team member's payout is recorded.

- **Step 1 — Client onboarding:** The admin adds a client and enters the basic contact information. The new client can then be selected when creating a task.
- **Step 2 — Task creation:** The admin creates a task, selects the team member who will handle it, sets its priority and due date, and enters the amount that will be paid for the work.
- **Step 3 — Team member view:** The assigned team member logs in and sees the task on their personal task board. They do not see tasks assigned to other team members.
- **Step 4 — Marking work complete:** After finishing the work, the team member changes the task status to Completed. This allows the task to move into the invoicing process.
- **Step 5 — Invoice generation:** The admin can see that the completed task is ready to be invoiced. By generating the invoice, the system uses the task amount automatically instead of requiring the admin to type it again.
- **Step 6 — Marking the invoice paid:** After the client makes the payment, the admin changes the invoice status to Paid.
- **Step 7 — Automatic payout update:** When the invoice becomes Paid, the system creates the related financial transaction and updates the team member's payout history and total earnings automatically.

This final step is an important part of the project because it removes another manual record-keeping task. The admin does not need to separately enter the team member's payout after receiving the client payment; the payout is created as part of the invoice payment process.

8. Challenges Faced & Solutions Implemented

The Supabase Problem

The first working version of CashTrack used Supabase for authentication and database storage. This caused the biggest technical difficulty during the project. The row-level security rules sometimes blocked requests that should have been allowed for the admin, and I had to spend time checking whether each problem was related to permissions, authentication sessions, or my own queries. I also found that environment variables that worked on my computer could cause authentication problems after the project was set up again. Since the deadline was getting closer, I decided that continuing with the backend was adding more risk than value for this project. I changed the data layer to LocalStorage, which removed the backend and the related configuration problems and made the application more dependable for the final demo.

Managing Global State as a Beginner

In the early version of the application, I passed task data through four or five levels of components just to reach the component that needed it. Changes also required callback functions to be passed back through the same chain. Although this worked, it became difficult to maintain as more features were added. I solved this by moving shared data into React Context and accessing it through custom hooks such as useTasks, useAuth, and useInvoices. This reduced the amount of unnecessary props and made the components easier to understand.

Keeping LocalStorage Data Consistent

Because tasks, invoices, and payouts were stored separately in LocalStorage, there was a chance that their information could become inconsistent. For example, an invoice could still contain information about a task that had later been removed. To reduce this problem, I kept the write operations inside the data-access layer. I also stored a copy of the important task details inside invoice and payout records when they were created. This means that later changes to a task do not unexpectedly change an existing financial record.

9. Learning Outcomes

Before starting this project, I had only limited experience with React from a few tutorials. Building the complete application was different because I had to make decisions about the structure and then solve the problems that appeared during development. The following points were the main things I learned from the project.

- I understood component architecture much better after having to decide which parts of the interface should be separate components. I also learned that making a component too large or splitting it unnecessarily can both make the code harder to manage.
- State management became much clearer when I faced the actual problem of passing task data through many components. Using React Context for shared data made more sense to me after solving this problem in the project instead of only reading about it in a tutorial.
- I also learned how to turn a business rule into application logic. For example, the rule that a paid invoice should increase a team member's earnings required me to understand how tasks, invoices, and payouts are related.
- Changing the original technical plan was another important lesson. Instead of spending all my time trying to force Supabase to work, I learned that it can be better to reconsider a decision when it is creating more problems than it solves.

10. Future Enhancements

CashTrack currently works as a self-contained project demo, but there are several features I would like to add if I continue developing it.

- Add PDF invoice export so that an admin can create an actual invoice file and send it to a client instead of only viewing the invoice inside the application.
- Add email or in-app notifications for task assignments and invoice status changes so team members can receive updates without constantly checking the dashboard.
- Connect the application to a proper backend, most likely Supabase, so data can be shared between devices. If I use Supabase again, I would plan the authentication and row-level security structure earlier in the development process.
- Add recurring invoices for clients who have ongoing retainer work instead of creating every invoice as a separate one-time task-based invoice.
- Add a CSV export for payout history so team members can keep a copy of their earnings outside the application.

11. Conclusion

CashTrack began with a simple problem: keeping track of the connection between completed freelance work and the payment that follows it. While building the project, it became much more than a dashboard because it gave me the chance to learn React by working on a complete application. Some parts of the final system were different from my original plan, especially the move from Supabase to LocalStorage and the decision to create the SVG charts myself. These changes happened because I wanted the application to work reliably for the final demonstration while also learning from the development process. The final result is a small system where a completed task can move through invoice creation and then update the related team member's payout. Working on CashTrack also helped me understand component structure, state management, data relationships, and the importance of making practical technical decisions. Overall, this project gave me a much better understanding of what is involved in building a real React application from start to finish.